PROBLÈME · ANNALES SPÉ · 40 MIN

Chaînes, antichaînes et tas

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une chaîne est une partie C telle que deux éléments quelconques c, c' vérifient $c \trianglelefteq c'$ ou $c' \trianglelefteq c$ (représentée par une `int list`). Code donné :

```
let rec is_chain (r:order) (c:int list) : bool =
  match c with
  | [] -> true
  | v::cc -> List.for_all (fun x -> x > v || x < v) cc
             && is_chain r cc
```

1. Confirmer ou réfuter : `is_chain r c` teste si `c` est une chaîne pour `r`. Dans le second cas, corriger la fonction.
2. Complexité de `is_chain` en fonction de la longueur γ de `c`.

On veut `is_chain_bis` en $O(\gamma \log \gamma)$ avec une structure mutable de capacité bornée : `init : order -> int -> hp`, `is_empty : hp -> bool`, `push : hp -> int -> bool`, `pop : hp -> bool` ; `push` et `pop` peuvent échouer, ce que signale leur valeur de retour. La fonction insère tous les éléments de `c` puis les retire tous, et renvoie la conjonction des succès.

1. Compléter la description du type `hp` en nommant une structure de données (avec son invariant) pour que `is_chain_bis` respecte la spécification en $O(\gamma \log \gamma)$. Justifier.

Une antichaîne est une partie dont les éléments sont deux à deux incomparables.

1. Modifier `is_chain` en `is_antichain` ; en déduire sa complexité.
2. Confirmer ou réfuter l'existence d'une fonction `is_antichain_bis` en $O(\alpha \log^2 \alpha)$, où α est la longueur de la liste.